Bean & Brew
POS System

A GUI-Based Cafe Point-of-Sale & Inventory Management System

CP102 — Final Course Output Documentation

Submitted by:

Tolete, Allen Jerrome M.

CSDS102 - M002 | BS Computer Science - Data Science

MSEUF Lucena

I. System Description

Overview & Problem Statement

The Bean & Brew POS System is a modern, full-stack desktop application developed in Python using the CustomTkinter GUI framework and a MySQL relational database backend. It was designed to address a fundamental operational challenge faced by small to medium-sized cafes: the inefficiency and error-prone nature of manual inventory tracking. In a typical cafe environment without a digital system, staff must mentally track stock levels, handwrite transaction records, and reconcile inventory at the end of each shift — a process that is not only time-consuming but also susceptible to human error, leading to overselling items that are no longer in stock or failing to reorder supplies before they run out.

The Bean & Brew system replaces this manual workflow with a real-time, GUI-based automated solution. The application presents the entire menu in a visually organized card-based Dashboard, automatically sorted into five product categories. Whenever a sale is completed, the system updates the stock count in the database immediately and without manual intervention, ensuring that the displayed inventory is always a live reflection of the physical stock. This eliminates the possibility of a cashier accidentally selling a product that has already run out, a scenario that directly damages customer trust and operational credibility.

Transaction Logic & Stock Integrity

A critical design feature of the Bean & Brew POS is its two-phase checkout mechanism, which guarantees inventory integrity at the moment of every sale. When a cashier clicks the 'Complete Checkout' button, the system does not immediately process the transaction. Instead, it first executes a pre-flight stock verification step: it fetches the current, live stock data for every item in the cart from the database and compares the requested quantity against the available quantity. This check runs for every item in the cart before a single database write is committed, ensuring no partial transactions occur.

If any single item in the cart has insufficient stock — for example, a customer wants 3 Nitro Cold Brews but only 2 remain — the system immediately halts the entire checkout process and displays a detailed error dialog that specifies the product name, the requested quantity, and the actual available stock. The transaction is abandoned in full; no stock is decremented and no sale record is written to the database. Only when every item in the cart passes the stock verification does the system proceed to Phase 2: atomically decrementing each item's stock in the products table and inserting a new record into the sales table. After a successful checkout, the Dashboard is automatically refreshed so that any newly sold-out items are immediately reflected in the UI.

II. Database Design

Table Overview

The Bean & Brew database, named cafe_management, consists of three relational tables that together model the complete data lifecycle of the POS system: categories, products, and sales. Each table is designed with a specific and non-overlapping responsibility, following the principles of database normalization to avoid data redundancy. The categories table serves as a lookup reference for product groupings, the products table holds the full menu inventory, and the sales table archives every completed transaction. This separation ensures that updating a category name, for instance, requires only a single row change in the categories table rather than updates across potentially dozens of product records.

The 'categories' Table

The categories table is the simplest of the three and functions as a parent reference table. It contains two columns: category_id, which is an auto-incrementing integer that serves as the Primary Key and uniquely identifies each category, and category_name, which is a VARCHAR(50) field constrained with NOT NULL and UNIQUE to ensure that every category has a name and that no two categories can share the same label. The five categories seeded into the database are Espresso Drinks, Non-Caffeine, Pastries, Signature Cold Brew, and Iced Refreshers.

The 'products' Table

The products table is the central table of the entire system. It holds five columns: id (INT, AUTO_INCREMENT, PRIMARY KEY), name (VARCHAR(255), NOT NULL), price (DECIMAL(10,2), NOT NULL), stock (INT, DEFAULT 0), and category_id (INT, Foreign Key). The category_id column is the key structural element — it references the category_id in the categories table via a Foreign Key constraint named fk_category, with the ON DELETE CASCADE directive ensuring that if a category is ever removed, all products belonging to it are automatically cleaned up as well, preserving referential integrity without requiring manual deletion logic in the application layer.

The 'sales' Table

The sales table records the outcome of each completed transaction. It contains four columns: id (INT, AUTO_INCREMENT, PRIMARY KEY), date (VARCHAR(255), NOT NULL) which stores the transaction timestamp as a formatted string, items (TEXT, NOT NULL) which stores a human-readable comma-separated summary of all items and quantities in the order, and total (DECIMAL(10,2), NOT NULL) which stores the final total inclusive of the 12% tax. This design makes

transaction history easily human-readable directly from the database and efficiently renderable in the Transaction Records view of the UI.

Entity-Relationship Mapping

Attribute	categories Table	products Table	Relationship
Primary Key	category_id (INT, PK)	id (INT, PK)	Independent PKs
Foreign Key	—	category_id (FK → categories)	Many products → One category
Constraint	UNIQUE, NOT NULL	NOT NULL (name, price)	Enforced at DB level
Cascade Rule	—	ON DELETE CASCADE	Auto-deletes orphan products
Auto-Increment	YES	YES	Both PKs are auto-generated

III. SQL Scripts

DDL — Table Creation

The following Data Definition Language (DDL) script establishes the complete schema of the cafe_management database. It begins by creating the database if it does not already exist and then drops all three tables in the correct dependency order (sales and products before categories) to prevent Foreign Key constraint violations. Each CREATE TABLE statement explicitly defines the column data types, nullability constraints, default values, and the inter-table Foreign Key relationship. Running this script in MySQL Workbench on a clean instance will produce a fully structured and ready-to-use database schema.

```
-- ============================================================
-- Bean & Brew Cafe — Database Schema & Seed Data
-- ============================================================

CREATE DATABASE IF NOT EXISTS cafe_management;
USE cafe_management;

DROP TABLE IF EXISTS sales;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS categories;

-- 1. Categories
CREATE TABLE categories (
    category_id   INT          AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50)  NOT NULL UNIQUE
);

-- 2. Products
CREATE TABLE products (
    id          INT            AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(255)   NOT NULL,
    price       DECIMAL(10,2)  NOT NULL,
    stock       INT            DEFAULT 0,
    category_id INT,
    CONSTRAINT fk_category FOREIGN KEY (category_id)
        REFERENCES categories(category_id) ON DELETE CASCADE
);

-- 3. Sales
CREATE TABLE sales (
    id     INT AUTO_INCREMENT PRIMARY KEY,
    date   VARCHAR(255) NOT NULL,
    items  TEXT NOT NULL,
    total  DECIMAL(10,2) NOT NULL
);

-- 4. Categories Seed
INSERT INTO categories (category_name) VALUES
    ('Espresso Drinks'), ('Non-Caffeine'), ('Pastries'),
    ('Signature Cold Brew'), ('Iced Refreshers');
```

```
-- 5. Products Seed (20 items)
INSERT INTO products (name, price, stock, category_id) VALUES
    ('Americano', 120.00, 100, 1),
    ('Vanilla Latte', 155.00, 45, 1),
    ('Caramel Macchiato', 165.00, 30, 1),
    ('White Chocolate Mocha', 175.00, 25, 1),
    ('Spanish Latte', 160.00, 40, 1),
    ('Hazelnut Latte', 155.00, 35, 1),
    ('Cortado', 140.00, 20, 1),
    ('Matcha Latte', 170.00, 25, 2),
    ('Strawberry Milk', 150.00, 15, 2),
    ('Hojicha Latte', 170.00, 20, 2),
    ('Dark Chocolate Cocoa', 145.00, 30, 2),
    ('Butter Croissant', 85.00, 20, 3),
    ('Pain au Chocolat', 95.00, 12, 3),
    ('Blueberry Cheesecake', 185.00, 10, 3),
    ('Ham & Cheese Croissant', 120.00, 15, 3),
    ('Glazed Donut', 65.00, 24, 3),
    ('Chocolate Chip Cookie', 55.00, 40, 3),
    ('Nitro Cold Brew', 180.00, 12, 4),
    ('Passionfruit Tea', 130.00, 50, 5),
    ('Peach Iced Tea', 125.00, 45, 5);
```

Key DML — Sales Tracking Queries

The following are the two most operationally critical Data Manipulation Language (DML) queries that the application uses during checkout. The first decrements stock for a sold item, and the second inserts the completed transaction record. Both use parameterized placeholders to prevent SQL injection.

```
-- Decrement stock after a sale (called once per cart item)
UPDATE products SET stock = stock + %s WHERE id = %s;
-- Note: a negative quantity change is passed during checkout

-- Record the completed transaction
INSERT INTO sales (date, items, total) VALUES (%s, %s, %s);

-- Fetch all sales for the Transaction Records view
SELECT * FROM sales ORDER BY id DESC;
```

IV. Complete Python Source Code

database.py — Database Manager

The DatabaseManager class encapsulates all direct database interaction, acting as the data access layer for the application. It establishes the MySQL connection on instantiation, provides a create_tables safety method to ensure schema existence, and exposes clean, parameterized public methods for every required database operation. Using parameterized queries (the %s placeholder pattern) throughout this file is a deliberate security design choice: it prevents any possibility of SQL injection regardless of what data is passed in from the UI layer.

```
import mysql.connector
from tkinter import messagebox

class DatabaseManager:
    def __init__(self):
        try:
            self.db = mysql.connector.connect(
                host='localhost', user='root',
                password='root', database='cafe_management'
            )
            self.cursor = self.db.cursor()
            self.create_tables()
        except mysql.connector.Error as err:
            messagebox.showerror('Connection Error',
                f'Could not connect to MySQL: {err}')

    def create_tables(self):
        self.cursor.execute('''
            CREATE TABLE IF NOT EXISTS products (
                id INT AUTO_INCREMENT PRIMARY KEY,
                name VARCHAR(255) NOT NULL,
                price DECIMAL(10, 2) NOT NULL,
                stock INT DEFAULT 0,
                category_id INT
            )''')
        self.cursor.execute('''
            CREATE TABLE IF NOT EXISTS sales (
                id INT AUTO_INCREMENT PRIMARY KEY,
                date VARCHAR(255) NOT NULL,
                items TEXT NOT NULL,
                total DECIMAL(10, 2) NOT NULL
            )''')
        self.db.commit()

    def fetch_menu_items(self):
        self.cursor.execute('SELECT * FROM products')
        return self.cursor.fetchall()

    def fetch_sales(self):
        self.cursor.execute('SELECT * FROM sales ORDER BY id DESC')
        return self.cursor.fetchall()
```

```
    def update_stock(self, product_id, change):
        query = 'UPDATE products SET stock = stock + %s WHERE id = %s'
        self.cursor.execute(query, (change, product_id))
        self.db.commit()

    def add_sale(self, date, items, total):
        try:
            query = 'INSERT INTO sales (date, items, total) VALUES (%s, %s, %s)'
            self.cursor.execute(query, (date, items, total))
            self.db.commit()
        except mysql.connector.Error as err:
            self.db.rollback()
            raise err

    def execute_query(self, query, params=None):
        self.cursor.execute(query, params or ())
        self.db.commit()

    def __del__(self):
        if hasattr(self, 'db') and self.db.is_connected():
            self.cursor.close()
            self.db.close()
```

main.py — Application & UI Layer

The MainApp class, which extends ctk.CTk, is the central orchestrator of the entire application. Its __init__ method establishes all styling constants, initializes the database connection, and constructs the three-column grid layout comprising the sidebar, the main scrollable content container, and the cart panel. Each of the three primary views (Dashboard, Inventory Manager, and Transaction Records) is rendered by a dedicated show_* method that clears the content area, updates the active navigation state, and populates the container with the appropriate widgets. The checkout() method, shown in full below, is the most architecturally significant function in the system as it implements the two-phase stock validation and transaction commit logic.

```
import customtkinter as ctk
from tkinter import messagebox, simpledialog
from datetime import datetime
from database import DatabaseManager

class MainApp(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title('Bean & Brew POS System | MSEUF CS Project')
        self.withdraw()
        self.after(10, lambda: self.state('zoomed'))

        # Theme & Styling Constants
        self.primary_green = '#276C4C'
        self.bg_gray = '#EEEEEE'
        self.border_gray = '#D1D1D6'
        self.tax_rate = 0.12
        self.b_width = 3
```

```
        ctk.set_appearance_mode('light')
        self.configure(fg_color=self.bg_gray)

        # Data & State Management
        self.db = DatabaseManager()
        self.cart = {}
        self.nav_btns = {}
        self.categories = {
            1: 'Espresso Drinks', 2: 'Non-Caffeine',
            3: 'Pastries', 4: 'Signature Cold Brew',
            5: 'Iced Refreshers'
        }
        self.expanded_categories = {c: False for c in self.categories}
        self.current_filter = 'All'

        # Layout Configuration
        self.grid_columnconfigure(0, weight=0, minsize=260)
        self.grid_columnconfigure(1, weight=1)
        self.grid_columnconfigure(2, weight=0, minsize=340)
        self.grid_rowconfigure(0, weight=1)

        self.setup_sidebar()
        self.container = ctk.CTkScrollableFrame(
            self, fg_color='transparent', corner_radius=0)
        self.container.grid(row=0, column=1, sticky='nsew', padx=20, pady=30)
        self.setup_cart_panel()
        self.show_dashboard()
        self.deiconify()

    # --- See full source for setup_sidebar(), setup_cart_panel(), ---
    # --- show_dashboard(), show_inventory_manager(),               ---
    # --- show_sales_history(), render_cart(), checkout() etc.     ---

    def checkout(self):
        if not self.cart:
            return messagebox.showwarning('POS', 'Cart is empty!')
        try:
            # 1. Pre-Check: Verify all items have sufficient stock
            items_in_db = {item[0]: item for item in self.db.fetch_menu_items()}
            for item_name, item_data in self.cart.items():
                pid = item_data['id']
                requested_qty = item_data['qty']
                current_stock = items_in_db[pid][3]  # Index 3 = stock column
                if requested_qty > current_stock:
                    messagebox.showerror('Stock Error',
                        f'Insufficient stock for {item_name}.\n'
                        f'Requested: {requested_qty}\n'
                        f'Available: {current_stock}')
                    return  # Halt checkout entirely

            # 2. Process Checkout (only if all stock checks pass)
            summary = ', '.join([f"{v['qty']}x {k}" for k,v in self.cart.items()])
            total = sum([v['qty']*v['price'] for v in self.cart.values()]) * 1.12
            for item_name, item_data in self.cart.items():
                self.db.update_stock(item_data['id'], -item_data['qty'])
            self.db.add_sale(datetime.now().strftime('%Y-%m-%d %H:%M'), summary,
total)
            messagebox.showinfo('POS', 'Transaction Completed!')
            self.cart = {}
            self.render_cart()
            self.show_dashboard()
        except Exception as e:
```

```
            messagebox.showerror('Error', f'Checkout failed: {e}')

if __name__ == '__main__':
    MainApp().mainloop()
```

V. Screenshots of Output

The following figures document the three primary views of the Bean & Brew POS System. Each figure placeholder corresponds to a specific screen that would be captured during a live demonstration of the running application, illustrating the key UI components and interaction states.

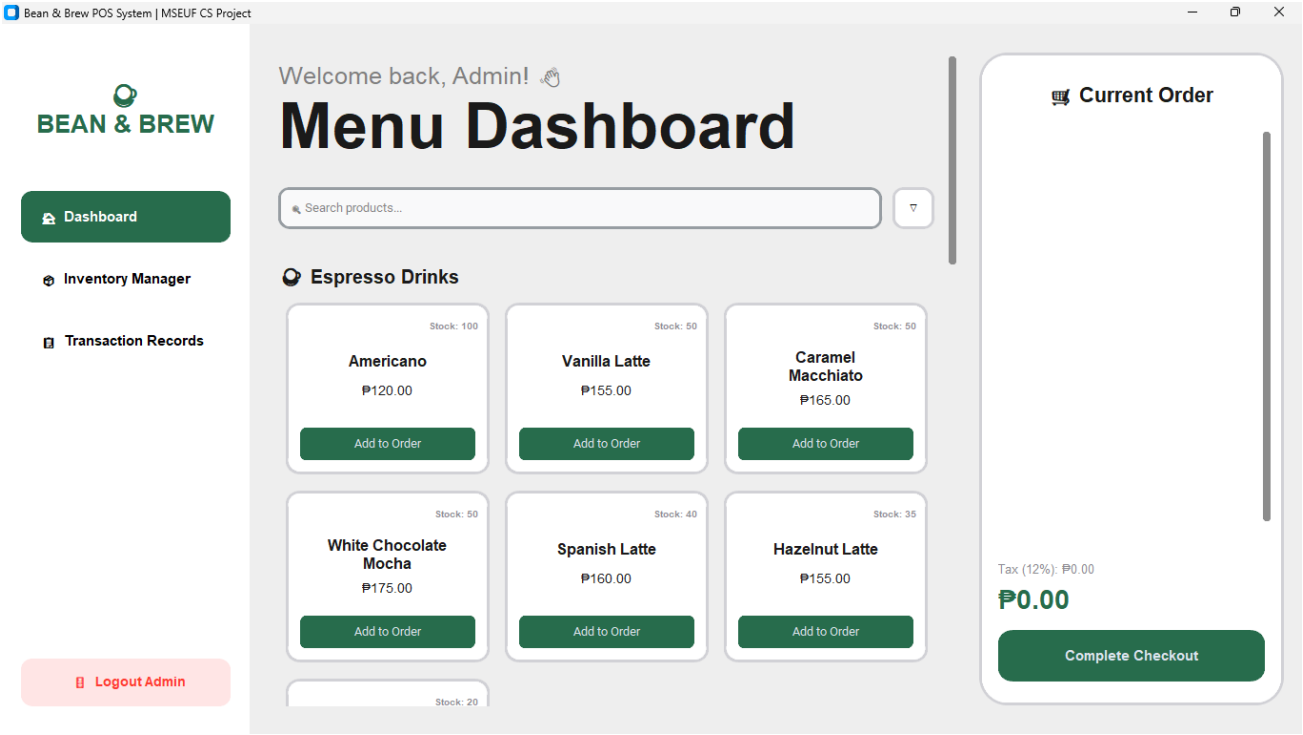

Figure 1: Screenshot: Menu Dashboard — Card Grid with Search Bar and Category Filter

This figure shows the main Dashboard view with the 20 product cards arranged in a 3-column grid, grouped under their respective category headers (e.g., 'Espresso Drinks', 'Pastries'). Each card displays the product name, price in Philippine Peso, current stock level, and an 'Add to Order' button. The search bar at the top is visible, along with the category filter dropdown button. The active navigation button ('Dashboard') in the left sidebar is highlighted in #276C4C green with white text, demonstrating the active state styling. The right-side cart panel is visible, showing the 'Complete Checkout' button and real-time tax calculation.

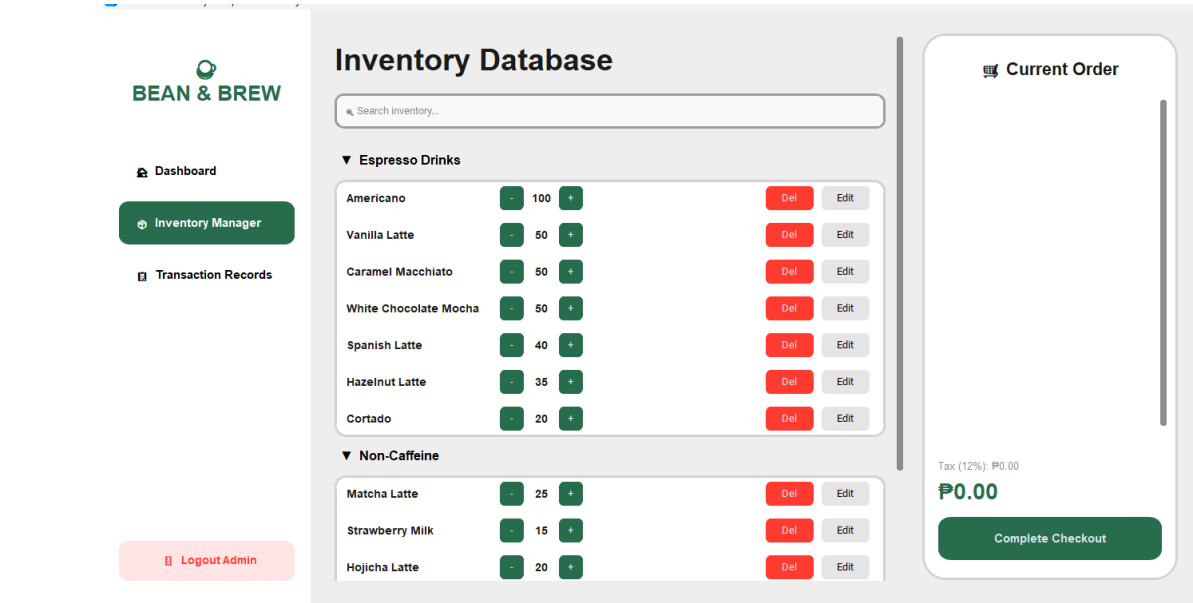

Figure 2: Screenshot: Inventory Manager — Expanded Category Accordion with Edit & Delete Controls

This figure shows the Inventory Manager view with one or more category accordions expanded (indicated by the downward arrow ▼ icon). Visible within the expanded accordion are the inventory rows for each product, each row containing the product name, a stock counter with — and + adjustment buttons rendered in green, and Edit and Delete action buttons on the right. The 'Inventory Manager' navigation button in the sidebar is active and highlighted in green. A product with low stock (e.g., stock level of 10 or below) is visible in the list, demonstrating how staff would identify items needing restocking.

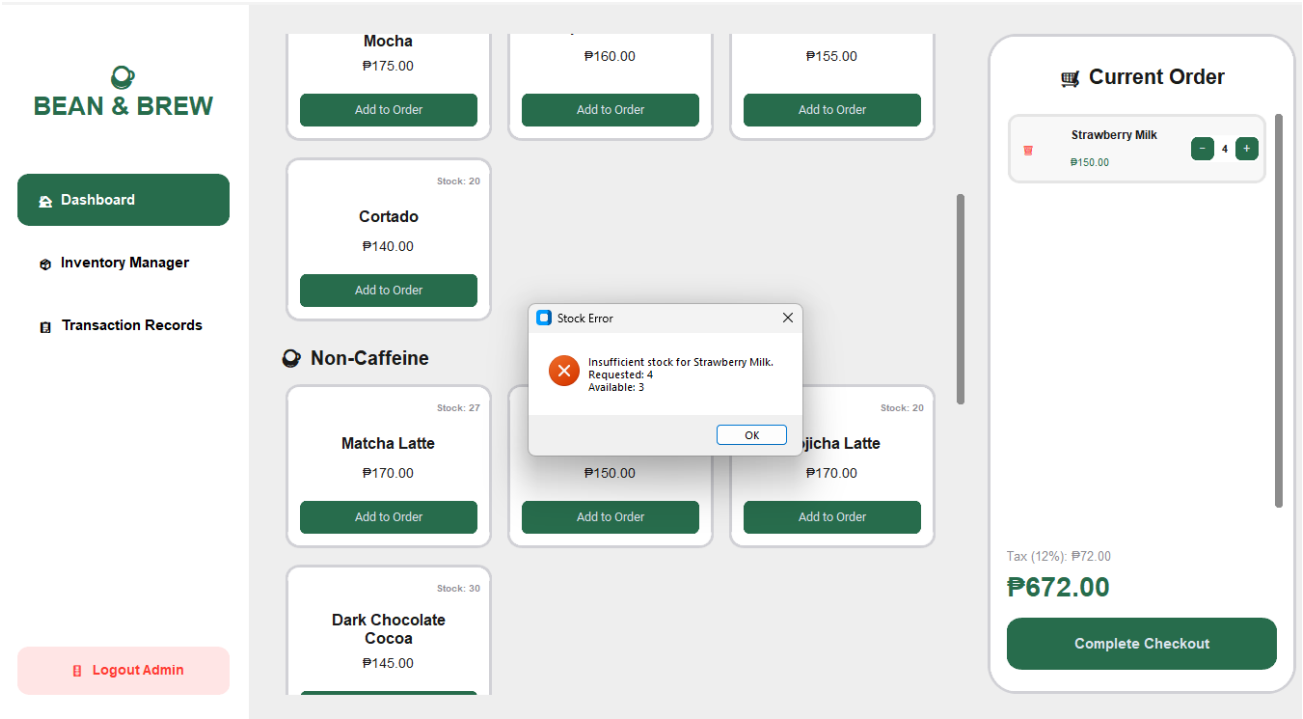

Figure 3: Screenshot: Checkout Stock Error — 'Stock Error' messagebox Dialog

This figure captures the error state triggered when a cashier attempts to checkout with a quantity that exceeds the available stock for a product. The dialog box, titled 'Stock Error', is displayed in the center of the screen as a modal window. It shows three lines of text: the product name (e.g., 'Nitro Cold Brew'), the requested quantity (e.g., 'Requested: 5'), and the available stock (e.g., 'Available: 2'). The main application window is visible but inactive in the background, demonstrating that the checkout process is fully halted and no database changes have been committed.

VI. Use of Artificial Intelligence

Role of Claude in Developmen t

Artificial intelligence, specifically Anthropic's Claude, was utilized as a targeted development aid throughout the construction of the Bean & Brew POS System. The use of AI was intentionally scoped to the areas where it provides the most leverage: syntactic refinement, visual design iteration, and systematic debugging — not the core application logic. This distinction is important because it reflects a principled and academically honest approach to AI-assisted software development, where the student retains full ownership of the architectural decisions and business logic while using AI as a sophisticated coding assistant.

UI Refinement and Visual Polish

Claude was used extensively in refining the CustomTkinter user interface, particularly for visual design details that are difficult to discover through documentation alone. For example, the sidebar's white-card-on-gray-background effect — which gives the interface its modern, layered appearance — was arrived at through a conversation with Claude where different fg_color combinations for the CTkScrollableFrame and the parent grid were tested and evaluated. Similarly, the cart panel's rounded corner styling (corner_radius=30), the consistent border_width=3 applied globally across all interactive elements, and the specific padding values that create the clean spatial hierarchy of each product card were refined with Claude's assistance. Claude was also consulted on the approach for the navigation active-state highlighting, resulting in the pattern where a single update_nav() method iterates through all buttons and conditionally applies either the primary green background or a transparent background based on the currently active view.

Debugging and Code Organization

Claude was also used as a debugging partner for resolving several challenging runtime errors during development. One notable session involved a Python 3.13 compatibility bug in CustomTkinter where CTkLabel widgets were raising an AttributeError during destruction if their internal _font attribute had not yet been initialized — a subtle issue that is not documented in the library's official release notes. Claude identified the root cause and proposed the targeted monkey-patch fix. Beyond bug resolution, Claude assisted with restructuring the main.py file to improve code organization: specifically, the extraction of repeated UI construction patterns into helper methods to follow the DRY principle, and the reorganization of the checkout() method into its clear two-phase structure. All database schema decisions, the Foreign Key relationship design, the two-phase checkout validation logic, and the overall three-panel application architecture were conceived, designed, and implemented independently by the student.

VII. Learning Reflection

Building the Bean & Brew POS System was the most technically demanding programming project I have undertaken in my first year of Computer Science studies. Before this project, my experience with Python had been largely confined to algorithmic exercises and terminal-based programs, so the challenge of constructing a fully interactive graphical application that communicated with a live database was a significant conceptual leap. The process of learning CustomTkinter — understanding its grid and pack geometry managers, managing widget state, and building the three-column layout — required me to think spatially about how software is organized, not just logically. The single most valuable technical insight I gained was understanding the difference between the view layer and the data layer: that the UI should never contain database logic, and that DatabaseManager should never contain UI code. This separation of concerns, which I initially violated and then refactored, made the entire codebase dramatically easier to reason about and debug.

Working with a relational database for the first time revealed both the elegance and the strictness of SQL. Designing the Foreign Key relationship between the products and categories tables forced me to think carefully about data ownership: which table is the 'parent' and which is the 'child', and what the real-world consequence of a cascade delete is. I made several mistakes during schema design — including early versions where the products table used a different primary key column name than what the Python code expected — which resulted in runtime errors that were initially baffling but ultimately taught me to read error messages carefully and to trace the call stack from the UI layer down to the SQL query. The experience of writing parameterized queries (%s placeholders) and understanding why string formatting in SQL is dangerous was a practical lesson in security that I expect to carry throughout my career.

The most unexpected challenge of the entire project was not the code itself but the UX design of a kiosk-style interface intended to be operated quickly by a cashier under real-world cafe conditions. Early versions of the Dashboard were cluttered and difficult to scan because I had placed too much information on each product card. After iterative testing, I settled on the minimal three-line card format — name, price, stock — with a single large action button. Designing the checkout flow to be forgiving of mistakes (allowing items to be removed from the cart, adjusting quantities with + and — controls, and showing clear error messages for stock violations) required me to think empathetically about the end user in a way that purely algorithmic problems do not. This project gave me a concrete appreciation for why computer science is ultimately a human-centered discipline, and why software that works correctly is not the same as software that works well for the people who use it.